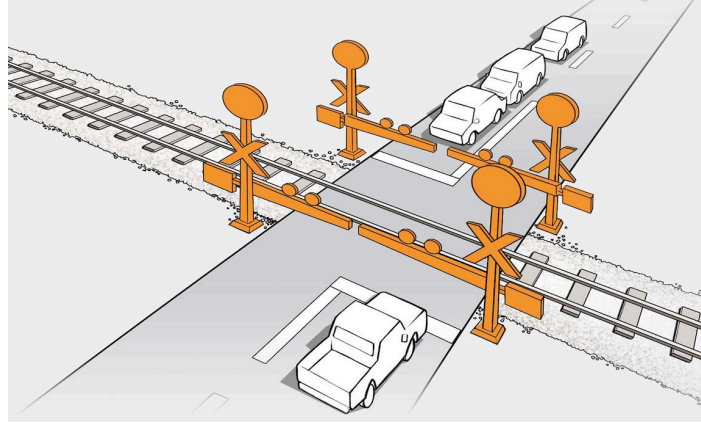


Modelling and verifying behaviour in mCRL2

Railway level crossing

System Verification 2026/2027, José Proença, FCUP, Portugal



A road crosses a railway track. The crossing is protected by a gate (barrier) that must be down while a train passes. A controller coordinates the trains and the gate. Time is abstracted away in this part: you model only the order of events, not durations.

Deliverables

This is a group assignment. You are expected to **work in pairs**, and submit a single report and a single set of mCRL2 files. More information below.

- The **.mcr12** files for Tasks 1 and 2 below (include also the faulty variants in Task 1e. and Task 2c.).
- The **.mcf** files with your formulas.
- A short **.pdf** report (about 3 pages) with the explanations requested in the tasks below.
- **Send by email** (jose.proenca@fc.up.pt) a unique zip file “SV2627-N1-N2.zip”, where N1 and N2 are your student numbers. The subject of the email should be “[SV] mCRL2 assignment”.
- **Deadline:** 9 Nov 2026 (Monday) @ 23h59

Task 1: One train (no data)

1a. Components

Model three processes:

- **Train:** approaches, waits for permission, occupies the crossing, leaves, notifies it left, and repeats.
- **Gate:** can be open or closed, and moves between the two via lowering and raising.
- **Controller:** on a train’s approach, orders the gate to lower, and only after the gate confirms it is down grants the train permission to enter. When the train has left, it orders the gate to raise.

Suggested actions: **approach**, **lower**, **down**, **grant**, **enter**, **exit**, **leave**, **raise**, **up**.

1b. Composition

Combine the processes with **par**, **comm** and **allow**. Make sure that internal handshakes are hidden (**hide**) when you inspect the state space, and that you include visible actions not involved in internal handshakes.

1c. Inspection

Generate the LTS and view it. Next view the reduced LTS using *branching bisimulation*. **Report:** include a screenshot of the both LTS and explain what remains in the reduced LTS.

1d. Properties

Formulate the following in mCRL2's modal μ -calculus and check them. **Report:** explain your properties and the results.

- **Safety:** whenever the train is on the crossing, the gate is down. (Formulate this in terms of action sequences, e.g. *no **up** happens between **enter** and **exit**, and no **enter** happens unless a **down** occurred since the last **up**.*)
- **Reachability:** the gate can be raised again after every train passage.
- **Liveness:** every **approach** is eventually followed by **exit**.
- **Deadlock freedom:** the train can always **approach** the crossing, even if not immediately.

1e. Break it

Create a faulty variant in which the controller **grants** permission to cross immediately after it orders the gate to **lower**, without waiting for it to be **down**. Show that your safety property fails, and inspect the counterexample trace. **Report:** Explain in words what went wrong and why the property fails.

Task 2: Multiple trains (with data)

Now N trains share the same single track and the same crossing (start with $N = 3$). Each train has an identity, for example:

```
sort TrainId = struct t1 | t2 | t3;
```

The actions carry that identity: **approach**(t), **grant**(t), **enter**(t), **exit**(t), **leave**(t). The gate actions (**lower**, **down**, **raise**, **up**) stay as in Task 1.

The trains are independent: any train that is not currently approaching or on the crossing may approach at any time. Since there is one track, the controller grants permission to **at most one train at a time**.

2a. Model with data

Extend your Task 1 model as follows.

1. Make the train process **parameterised by its identity**, $\text{Train}(t: \text{TrainId})$, and compose the trains in parallel (either by writing the instances explicitly or by using a suitable parallel composition over all identities).
2. Write a controller that **uses data to keep track of the trains** that approached and did not yet left. Choose **one** of these designs as your main model:
 - **Counter model:** the controller stores $n: \text{Nat}$, the number of trains that have approached but not yet exited.
 - **Set model:** the controller stores $S: \text{FSet}(\text{TrainId})$ (see [documentation](#) – or a list, if you want to model a queue) with the trains that are present.
3. The controller must **keep the gate down** as long as at least one train is present, and must only raise it when none is.
4. **Report:** explain the invariant – argue why your controller maintains the invariant regarding the number (or set) of trains that have approached but not yet exited.

*Hint: keep the data bounded. A counter that can grow without limit may give an infinite state space. Guard the **approach** action, or make the bound a parameter N .*

2b. Properties

Formulate and check the following in mCRL2's modal μ -calculus, using data quantification where relevant (e.g. $\text{forall } t: \text{TrainId}. [\text{true}^* \ . \ \text{approach}(t)] \ \dots$).

1. **Safety:** whenever some train is on the crossing, the gate is down.
2. **Mutual exclusion:** two different trains are never on the crossing at the same time.
3. **No premature raising:** the gate is never raised while some train has approached but has not yet exited.
4. **Deadlock freedom:** There is always the possibility of (eventually) having a train approaching.
5. **Liveness per train:** for every $t: \text{TrainId}$, every **approach**(t) is eventually followed by **enter**(t).
Report: explain what were your fairness assumptions regarding trains that can approach infinitely often and may or may not enter.
6. **Road availability:** the gate can always eventually open.
7. **Road availability (infinite opening):** the gate is guaranteed to open infinitely often. *Report: Discuss what a counterexample means in practice, i.e., what it means if the property fails.*
8. **Non-vacuity:** the safety and no-premature-raising properties are only meaningful if their premises can actually occur. Check if their premises are reachable.

2c. Break it: a family of bugs

Implement each of the following faulty controllers as a separate variant. For each one, determine which of your properties starts to fail. *Report: give the shortest counterexample, and explain the scenario in words.*

1. **Naïve raise:** the controller raises the gate as soon as **any** train exits.
2. **Missed increment:** the counter (or set) is not updated when a train approaches while another train is already on the crossing.
3. **Off-by-one:** the decrement on **exit** is done incorrectly, e.g. the controller believes the section is empty one train too early, or never gets back to 0.

To reflect (no need to include in the report): which of these bugs could you even express in the data-free model of Task 1? Which properties detect which bug, and are there bugs that none of your properties detect?

2d. State space and abstraction

Report: present the number of states and transitions (before and after reduction modulo branching bisimulation) for $N = 1, 2, 3, 4$ and, if feasible, 5 and 6. Comment on the growth. Do you see symmetry between the trains?